

CS 499 Milestone Two Narrative
Giuseppe Claudio Zema
07/16/2026

The artifact selected for this enhancement is a JupyterDash animal rescue management system originally developed for CS 340 during C6-2025 (November/December) at SNHU. The original application displayed animal records stored in MongoDB and allowed users to filter available rescue animals through a web dashboard. While the original application successfully demonstrated database connectivity and interactive data visualization, it was limited to a single-page Python dashboard with minimal security, no user authentication, and limited software engineering practices.

For the CS 499 capstone, the artifact was redesigned into a modern MEAN-stack application consisting of an Angular frontend, an Express.js REST API, and a MongoDB database. This enhancement transformed the original classroom project into a more realistic web application that better reflects current software engineering practices.

I selected this artifact because it demonstrates the greatest amount of growth across multiple areas of software development. Rather than simply improving the appearance of the original application, I redesigned the overall architecture to create a modular client-server application that separates presentation, business logic, and data access into independent components. The frontend was rebuilt using Angular standalone components and reusable services, while the backend exposes RESTful API endpoints through Express. The application now supports full CRUD functionality for animal records, role-based user authentication, administrative tools, and improved database operations.

Several enhancements demonstrate software engineering and design principles. Authentication was implemented using JSON Web Tokens (JWT), allowing users to securely log into the application while protecting API endpoints through middleware. Authorization was added through role-based access control so that administrative functions are only available to authorized users. Angular route guards and an HTTP interceptor were implemented to automatically protect client-side routes and attach authentication tokens to outgoing requests. Although the frontend hides administrative features from unauthorized users, authorization is also enforced on the backend to ensure that manually crafted requests cannot bypass security controls.

The application was further expanded with a complete administrative interface that supports database maintenance operations. Administrators can create database backups, restore previous backups, download backup files, review audit logs, and navigate large administrative datasets through pagination. Audit logging records privileged operations such as creating, updating, deleting, backing up, and restoring data, providing accountability and traceability for administrative actions.

The dashboard itself was significantly improved to provide a more professional user experience. Users can search animal records across multiple fields, customize visible columns, filter by rescue type, navigate paginated datasets, display breed distribution charts generated through MongoDB aggregation pipelines, and interact with an integrated map. Separate forms were created for adding and editing animal records, including required-field validation and automatic calculation of an animal's age in weeks before records are submitted. Together, these improvements provide a significantly more complete and maintainable application than the original dashboard.

Software quality was also strengthened through automated testing. The Angular frontend includes unit tests written with Vitest for pages, the dashboard, and services. The Express backend includes Jest and Supertest integration tests covering authentication, authorization middleware, administrative routes, and API functionality. Adding automated tests increased confidence that new enhancements could be implemented without introducing regressions while also demonstrating professional software engineering practices.

The enhancement met the outcome-coverage plan established in Module One. Outcomes One and Two were fully addressed through the development of a professional-quality application that supports multiple user roles, administrative workflows, modular architecture, and clear technical communication through documentation, testing, and maintainable code organization. Significant progress was also made toward Outcomes Four and Five by implementing a modern MEAN architecture, reusable Angular services, RESTful APIs, middleware-based authentication and authorization, database administration utilities, and multiple layers of security. Outcome Three was intentionally reserved for the Algorithms and Data Structures enhancement that will follow in the next milestone, where optimization techniques such as caching and additional algorithmic improvements will become the primary focus. At this stage, no changes to the original outcome-coverage plan are necessary.

Working through this enhancement reinforced the importance of designing software as a collection of independent, reusable components rather than a single application. Converting the original JupyterDash project into a client-server architecture required careful planning of responsibilities between the frontend, backend, and database. I gained experience organizing large projects into reusable services, middleware, utility modules, and standalone Angular components while maintaining consistent communication between each layer of the application.

The most significant challenges involved implementing authentication, authorization, and administrative functionality while ensuring that the application remained secure and easy to maintain. Protecting routes on both the frontend and backend required careful coordination between Angular route guards, JWT middleware, role-based authorization, and the HTTP interceptor. Creating reliable database backup and restore utilities introduced additional challenges related to data validation and recovery, including correcting invalid imported records during the restore process. Developing automated tests also required learning how to isolate components and verify application behavior across both frontend and backend environments.

Overall, this enhancement transformed the original academic project into a significantly more complete software application that reflects modern software engineering practices. The resulting artifact demonstrates modular architecture, secure application design, reusable components, comprehensive testing, improved database administration, and an enhanced user experience. These improvements provide strong evidence of my growth as a software developer and represent an important addition to my professional ePortfolio.